MDQ Specification Audit: short-answer: mdq-spec

Date: 2026-10-17

Scope: Short-answer question type: `docs/question-types/short-answer.md`, `schema/short-answer.yaml`, its corpus in `examples/`, and the shared specs it depends on (`docs/references/grammar.md`, `docs/references/patterns.md`, `docs/question-types/question-base.md`, `docs/exam.md`, `docs/responses.md`).

Methodology

The audit covered the short-answer question type: docs/question-types/short-answer.md, the shared references it depends on (docs/references/patterns.md, docs/references/grammar.md), docs/question-types/question-base.md, docs/exam.md, docs/responses.md, the short-answer blanks of docs/question-types/fill-in.md, schema/short-answer.yaml and the short-answer corpus in examples/.

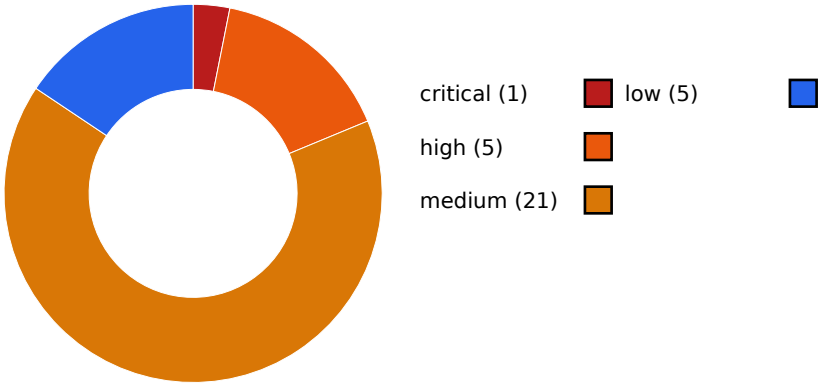
scripts/spec_audit.py passed all 14 steps before the audit. The lead auditor read the specs and probed edge cases with the mdq-py parser. A grammar subagent turned the Lark snippets into test grammars and checked what they accept and reject against the prose and the corpus. An independent scanner subagent read the type specification, the schema and the corpus, validated every complete example with `mdq validate`, and checked the Additional Rules table against the MUST, SHOULD and MAY requirements of the text.

Spelling and grammar fixes were made in place and are not findings.

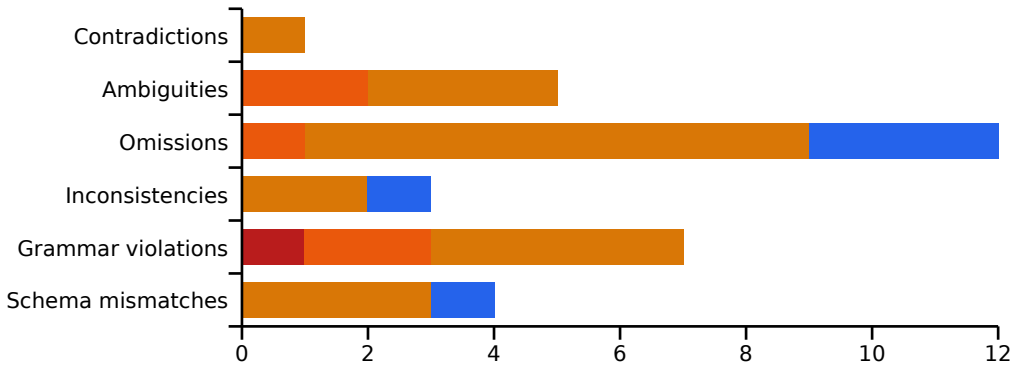
Executive summary

Severity	#
CRITICAL	1
HIGH	5
MEDIUM	21
LOW	5
INFO	0
Total findings	**32**

Findings by severity



Findings by category



Strengths and weaknesses

Strengths

✓ **Grammar violations:** The short-answer block grammar enforces the block-shape rules it covers. Tested with Lark. Accepted: `[short-answer]: Brasília`, `[short-answer/accept]: Brasília`, an exact literal with a trailing tab, an exact literal with inner spaces, an empty `[short-answer]:` with and without trailing ws, a list after the tag (LF, CR, CRLF, trailing ws before nl), `[short-answer/reject]: Rio`, a reject list, a bare * and inner backticks as inexact content. Rejected, as the rules require: an empty `[short-answer/reject]:`, accept and reject with a list after a blank line, content followed by a list, an unclosed backtick, `[short-answer/accept/reject]:` and `[short-answer/preAccept]:`. (docs/question-types/short-answer.md:149-160)

✓ **Grammar violations:** The common `ws/nl` rules hold for short-answer: tab after the colon is ws; CR, LF and CRLF all end the tag line; U+00A0 after the colon is read as text (accepted as PLAIN_TEXT, as grammar.md:44-46 states); U+00A0 before the tag and a four-space indent are not accepted as a tag. EXACT_TEXT and PLAIN_TEXT use only explicit classes, as the regex dialect section requires. (docs/references/grammar.md:9-46)

✓ **Omissions:** The grading rules of short answer are an ordered list with an explicit default for `unmatched`, and every branch (accept, reject, unmatched incorrect/manual, reject-only, no pattern) has a grading vector. (docs/question-types/short-answer.md:256-269, examples/grading/short-answer.yaml:96-163)

✓ **Ambiguities:** The inexact comparison is a numbered list of steps over versioned Unicode/CLDR tables, with a worked table and per-step vectors for both `diacritics` values. (docs/references/patterns.md:43-99, examples/matching/inexact.yaml)

✓ **Contradictions:** All nine complete markdown examples of short-answer.md, patterns.md and the fill-in short-answer blank section validate with mdq-py with no diagnostic, and the detached-list counterexample gets the documented `detached-answer-list` warning. (docs/question-types/short-answer.md:7-64, docs/question-types/short-answer.md:310-372, docs/references/patterns.md:25-29, docs/question-types/fill-in.md:244-254)

✓ **Omissions:** The parse-error rows of Additional Rules (repeated block, content followed by a list, empty reject, interleaved feedback) and the regex flag rows each have an invalid corpus example with the expected code. (examples/invalid/short-answer-repeated-block.lint.json, examples/invalid/short-answer-content-and-list.lint.json, examples/invalid/short-answer-empty-reject.lint.json, examples/invalid/short-answer-interleaved-observations.lint.json, examples/invalid/short-answer-repeated-regex-flag.lint.json)

Central risks

• The single-line form and the list form of a pattern block are specified in three places (prose, Lark snippet, patterns.md continuation rules) that disagree; mdq-py follows CommonMark, which none of them states.

• Regex semantics (Unicode mode, case folding, shorthand classes, subject normalization) are left to the host regex engine, so mdq-py and mdq-js will grade the same response differently.

• Several diagnostics exist only in the corpus and the implementation, not in the Additional Rules table, so the table cannot serve as the list of lint codes for short answer.

Detailed findings

Contradictions

Severity	Location	Description
MEDIUM	docs/responses.md:156-158	**F14** responses.md says every short answer is case folded before matching

F14: responses.md says every short answer is case folded before matching

Location: docs/responses.md:156-158

Mechanical fix

```
Short answers are normalized before matching -- case folded, Unicode
```

Description: Only inexact literals are case folded and have their whitespace collapsed. Exact literals keep case and interior whitespace (patterns.md:133-134), and a regex is case-sensitive unless it has `i` (patterns.md:281).

Impact: A reader of responses.md expects `` ` math.isnan `to accept` Math.isnan` ``.

Suggested fix: Say that each pattern normalizes the response in its own way, with a link to patterns.md.

Ambiguities

Severity	Location	Description
HIGH	docs/references/patterns.md:223-226, docs/question-types/short-answer.md:182-183	**F9** Single-line content: which lines continue the pattern is unclear
HIGH	docs/references/patterns.md:260-267, docs/references/patterns.md:304-305	**F11** Regex Unicode mode, case folding and shorthand classes are undefined
MEDIUM	docs/references/patterns.md:147-150, docs/references/patterns.md:31-32	**F12** Regex delimiters: an unescaped / in the body and spaces after the flags
MEDIUM	docs/references/patterns.md:289-294	**F13** The n flag strips diacritics from the regex source with no rule for escapes
MEDIUM	docs/references/patterns.md:166-167, schema/short-answer.yaml:108, docs/question-types/short-answer.md:388-392	**F25** Pattern feedback is said to come from the pattern that 'decided' the response, but the rule picks the first matching pattern that has feedback

F9: Single-line content: which lines continue the pattern is unclear

Location: docs/references/patterns.md:223-226, docs/question-types/short-answer.md:182-183

```
The content of a single-line block continues in the same way. The lines after
the tag line, up to the first blank line, continue the pattern:
```

Description: The text says every line after the tag line, up to the first blank line, continues the pattern. CommonMark lets a block quote, an ATX heading, a fenced code block, an HTML block or a thematic break interrupt a paragraph. mdq-py follows CommonMark: `[short-answer]: Brasília` followed by `> Good` gives `accept: [Brasília]` and `epilogue: "> Good"`, and the same for `## Heading` and `<div>x</div>`. Read literally, the spec makes the pattern `Brasília > Good`. For a list line the two texts contradict each other: by patterns.md, `[short-answer]: Brasília` followed by `* Rio de Janeiro` is the pattern `Brasília * Rio de Janeiro`, but short-answer.md:182-183 and row 414 make it a parse-error, as do mdq-py and examples/invalid/short-answer-content-and-list.mdq.md. A `>` line right after single-line content also looks like feedback, which short-answer.md:169-170 says the single-line form cannot have. A `!` line is treated differently again: `[short-answer]: Brasília` followed by `! note` gives the pattern `Brasília ! note` in mdq-py, so a comment silently changes the answer key. An empty tag followed by a text line (`[short-answer]:` then `Brasília`) gives `accept: [Brasília]` in mdq-py.

Impact: Two implementations can produce different answer keys from the same document. An author who writes feedback under a single-line answer gets it in the epilogue, shown to every student, with no diagnostic.

Suggested fix: State which lines end the single-line content: the same lines that LAZY_START excludes (grammar.md, Feedback and comment lines). Decide what a `>` or `!` line right after single-line content is: an error, a warning, or feedback for that pattern.

Exploitability conditions: A single-line `[short-answer]:` block followed, without a blank line, by a line that starts a block quote, heading, fence, HTML block or thematic break.

F11: Regex Unicode mode, case folding and shorthand classes are undefined

Location: docs/references/patterns.md:260-267, docs/references/patterns.md:304-305

```
* Unicode: Unicode characters are allowed in the regex. Unicode escape sequences
```

Description: The u flag is ignored, so it is not clear whether a regex runs in JavaScript Unicode mode. That decides if `.` matches one astral code point (`/./` vs an emoji), if `\u{1F600}` is valid, and how `i` folds case (`/s/i` vs `ſ`, `/k/i` vs U+212A KELVIN SIGN). The spec says `\d`, `\w`, `\s` have JavaScript semantics; mdq-py compiles with Python `re`, where `\d` matches `٣` and `\s` differs as grammar.md:11-15 itself describes. 'users SHOULD NOT assume UTF-16 encoding' is advice to authors, not a rule for implementations.

Impact: The same regex accepts different responses in mdq-py and mdq-js.

Suggested fix: Pick one mode (recommended: code points, as JavaScript u/v mode and Python `re` do) and state the semantics of `.`, `i` (simple case folding?) and the shorthand classes (ASCII only, as JavaScript). Add vectors to examples/ for each.

Exploitability conditions: A regex with `.`, `i`, `\d`, `\w` or `\s` and a non-ASCII response.

F12: Regex delimiters: an unescaped / in the body and spaces after the flags

Location: docs/references/patterns.md:147-150, docs/references/patterns.md:31-32

```
A regex must be closed by a `/` as well, and may include optional flags after
```

Description: The text does not say which / closes the regex. patterns.md:31 implies the last one (`/usr/bin` is `usr` with flags `bin`), and mdq-py accepts `/a/b/` as the body `a/b`, which is a syntax error in JavaScript. Leading spaces and tabs are stripped (line 147), but trailing ones are not: mdq-py rejects the item `* /abc/i ` with 'unknown regex flag: " "', while an exact literal allows them (EXACT rule `ws?`).

Impact: Authors cannot predict whether / needs escaping; implementations that follow JavaScript literal syntax reject documents mdq-py accepts.

Suggested fix: State that the last / closes the regex and that an unescaped / in the body is allowed (or forbidden, `invalid-regex`). State that spaces and tabs after the flags are stripped.

F13: The n flag strips diacritics from the regex source with no rule for escapes

Location: docs/references/patterns.md:289-294

```
* `n`: Strip diacritics from both the pattern and the response before
```

Description: Stripping diacritics from a regex source is not the same as from text. mdq-py normalizes only the literal characters written in the source: `/caf\u00e9/n` keeps é in the compiled regex and so never matches the stripped response `cafe`. A class such as `[\u0300-\u036f]` or a literal combining mark inside a class changes meaning after the strip. The spec says nothing about either case.

Impact: A regex with n can match nothing, with no diagnostic, and implementations can differ.

Suggested fix: Define n on the subject only (strip the response) and require the regex to be written without diacritics, or define how escapes are treated. A lint for a regex under n that still holds an `Mn` code point would catch the mistake.

F25: Pattern feedback is said to come from the pattern that 'decided' the response, but the rule picks the first matching pattern that has feedback

Location: docs/references/patterns.md:166-167, schema/short-answer.yaml:108, docs/question-types/short-answer.md:388-392

```
Only `pattern` is required. `feedback` is shown to the student whose response
```

Description: patterns.md and the schema say `feedback` is shown to the student whose response the pattern decided. short-answer.md:391-392 says the student gets the feedback of the FIRST pattern that matches AND defines a feedback message. With `accept: ["/cerrad[oa]/i", {pattern: Cerrado, feedback: Right}]`, the response `Cerrado` is decided by the first pattern, which has no feedback; by short-answer.md it shows `Right`, by patterns.md it shows nothing. 'Decided' is not defined for a response that several patterns match.

Impact: Implementations can show or omit feedback for the same response.

Suggested fix: In patterns.md and the schema, refer to the selection rule of short-answer.md ('Feedback') instead of 'the pattern that decided'.

Exploitability conditions: A response that matches two patterns of the deciding list, and only the later one has feedback.

Omissions

Severity	Location	Description
HIGH	docs/references/patterns.md:145-150	**F10** The response is not normalized or trimmed before a regex match

Severity	Location	Description
MEDIUM	docs/responses.md:148-153	**F15** Short-answer response: type, empty string and line breaks are not defined
MEDIUM	docs/question-types/short-answer.md:140-145, docs/question-types/short-answer.md:204-206	**F16** The required `[short-answer]` tag has no rule row and blocks the frontmatter-only form
MEDIUM	docs/question-types/short-answer.md:407-422	**F17** Additional Rules misses the pattern-string and preAccept-block rules
MEDIUM	docs/question-types/short-answer.md:158	**F18** An exact literal cannot contain a backtick
MEDIUM	docs/question-types/short-answer.md:104-115, docs/question-types/fill-in.md:57-60	**F26** Feedback of `preAccept` and `preReject` patterns has no rule
MEDIUM	docs/question-types/fill-in.md:202-204, docs/question-types/fill-in.md:280-286, schema/fill-in.yaml:120-128	**F27** Short answer blanks of fill-in have no `incorrectFeedback`, and the spec does not say so
MEDIUM	docs/question-types/short-answer.md:285-289, docs/question-types/short-answer.md:142-143, docs/question-types/short-answer.md:221	**F29** Additional Rules has no row for the `automation` key, the MAY permissions and the read-only property SHOULD
MEDIUM	examples/grading/short-answer.yaml:13	**F32** No corpus vectors for feedback selection, `incorrectFeedback`, pre-validation and the `f`/`b` flags
LOW	docs/exam.md:335-337	**F20** The exam `grading` table does not say what it does to short answer
LOW	docs/question-types/short-answer.md:409, docs/question-types/short-answer.md:420, docs/question-types/short-answer.md:421	**F31** Rows of Additional Rules with no text in the spec
LOW	docs/question-types/short-answer.md:256	**F34** GLOSSARY.md has no entry for pattern, settled, exact or inexact literal, or pre-validation

F10: The response is not normalized or trimmed before a regex match

Location: docs/references/patterns.md:145-150

```
A pattern is a regex if it starts with a `/` after stripping spaces and tabs.
```

Description: Exact literals define NFC plus trimming (patterns.md:126-131) and inexact literals define six steps, but no text says what happens to the response before a regex matches it. The implicit anchors make this visible: does `/1822-0?9-0?7/` accept `1822-09-07` (leading space) or `1822-09-07\n`? Does `/Brasília/` accept `Brasília` typed in NFD? mdq-py matches the raw response, so both fail. examples/grading/short-answer.yaml tests trimming only for literals.

Impact: A student who types a trailing space or newline, or whose input method produces NFD, fails a regex answer that a literal would accept. Implementations can disagree.

Suggested fix: Define the regex subject, for example: normalize the response to NFC and remove UNICODE_SPACE from both ends, as for exact literals. Add grading vectors for it.

Exploitability conditions: Any regex pattern, with a response that has surrounding whitespace or is not in NFC.

F15: Short-answer response: type, empty string and line breaks are not defined

Location: docs/responses.md:148-153

```
The text the student wrote, before normalization.
```

Description: The section does not say that a value that is not a string (`1960`, a list, a mapping) is a malformed response, as numeric and true/false do. It does not say how an empty string or a string of only spaces is graded: it is not `null`, so it is not skipped; examples/grading/short-answer.yaml grades `""` as pending only for a manual question. It does not say if the text can hold line breaks; inexact step 6 collapses them, a regex `.` never matches one.

Impact: A YAML response `1960` can be coerced to text in one implementation and refused in another. An empty submission scores 0 under `unmatched: incorrect` but its status is not specified.

Suggested fix: Add: the response is a string; any other value is a malformed response (`ResponseError`). State that an empty or whitespace-only string is graded like any other response (or is a skip). Add vectors.

F16: The required `[short-answer]` tag has no rule row and blocks the frontmatter-only form

Location: docs/question-types/short-answer.md:140-145, docs/question-types/short-answer.md:204-206

```
incorrect responses. A question MUST have a `[short-answer]` block and MAY
```

Description: The MUST has no row in Additional Rules and no lint code. mdq-py rejects a `.mdq.md` with `type: short-answer` and `accept` in the frontmatter and no tag ('parse-error: A required field is missing'). The author must write an empty `[short-answer]:` tag, which line 204 implies is allowed with an `accept` field but never states. The frontmatter field is called 'the canonical form' (short-answer.md:91), yet it cannot stand alone in markdown.

Impact: Authors who use the frontmatter form get an unexplained parse error, and the empty tag that fixes it looks like a 'no pattern' question.

Suggested fix: Either drop the requirement when `type` or `accept`/`reject` is in the frontmatter, or keep it and add a row (`parse-error`) plus a sentence that an empty tag with an `accept` field is valid.

F17: Additional Rules misses the pattern-string and preAccept-block rules

Location: docs/question-types/short-answer.md:407-422

```
| accept, reject, preAccept, preReject | error    | schema-error           | a pattern list,
when present, has at least one pattern (schema)                      |
```

Description: Rules stated in prose or enforced by the corpus have no row: a pattern string cannot be empty or blank (patterns.md:20; examples/invalid/short-answer-empty-pattern gives schema-error); content that opens with a backtick MUST be a complete span (short-answer.md:215-217; short-answer-partial-backtick gives schema-error); an empty pattern line in a list (short-answer-empty-pattern-line gives parse-error); a `[short-answer/preAccept]` block (short-answer-pre-accept-block gives forbidden-block-element, which the footnote at short-answer.md:104 does not mention). patterns.md:231 'Authors SHOULD write those on one line' has no warning code either.

Impact: The table is not the complete list of diagnostics; an implementer reading it misses four errors.

Suggested fix: Add one row per rule, with its code. Decide if the SHOULD at patterns.md:231 gets a warning code or becomes plain advice.

F18: An exact literal cannot contain a backtick

Location: docs/question-types/short-answer.md:158

```
EXACT_TEXT   : /[^`\n\r]+/
```

Description: EXACT_TEXT excludes the backtick and no escape exists, so an exact answer such as the shell command substitution `` `date` `` or the Markdown code `` `code` `` cannot be written. CommonMark solves this with longer backtick fences (`` ab` ``). mdq-py gives a schema-error with the raw JSON Schema regex for `` ` x ` ``.

Impact: Questions about shells, Markdown or SQL identifiers cannot use exact matching; the error message does not help.

Suggested fix: Accept a CommonMark code span of N backticks, with the same one-space stripping, or document the limitation and suggest a regex.

F26: Feedback of `preAccept` and `preReject` patterns has no rule

Location: docs/question-types/short-answer.md:104-115, docs/question-types/fill-in.md:57-60

```
systems should warn students about the invalidity of their answer before
```

Description: The pattern object form allows `feedback` in every pattern list, and fill-in.md:57-60 shows a `preReject` entry with the feedback 'Write the season, not a month number.' The Feedback section (short-answer.md:386-403) covers only `accept` and `reject`. No text says when a pre-validation feedback is shown, which one when several patterns match, or what a student sees when no `preAccept` pattern matches. The lowercase 'should warn' at line 107 is also not an RFC 2119 keyword, so the level of the requirement is unclear.

Impact: Authors write pre-validation feedback that implementations may never show, or show differently.

Suggested fix: Add a rule: when a submission is invalid, show the feedback of the first matching `preReject` pattern that has one; state what is shown for a `preAccept` failure (no pattern matched, so no pattern feedback). Write the warning requirement with SHOULD.

Exploitability conditions: A `preAccept` or `preReject` entry with `feedback`.

F27: Short answer blanks of fill-in have no `incorrectFeedback`, and the spec does not say so

Location: docs/question-types/fill-in.md:202-204, docs/question-types/fill-in.md:280-286, schema/fill-in.yaml:120-128

```
A short answer blank takes the same answer machinery as a standalone
```

Description: fill-in.md says a short answer blank takes the same answer machinery as a short answer question and shows feedback under the same rule. The short answer rule ends with `incorrectFeedback` for an incorrect response that no pattern gave feedback to (short-answer.md:399-403). The fill-in frontmatter table, the blank schema (`additionalProperties: false`, no `incorrectFeedback`) and the fill-in text do not offer it and do not say that blanks do without it.

Impact: A reader cannot tell whether an unmatched blank response can carry feedback; the 'same machinery' sentence suggests it can.

Suggested fix: State that blanks have no `incorrectFeedback`, or add the field (per blank or per question).

F29: Additional Rules has no row for the `automation` key, the MAY permissions and the read-only property SHOULD

Location: docs/question-types/short-answer.md:285-289, docs/question-types/short-answer.md:142-143, docs/question-types/short-answer.md:221

```
`automation` is not a field of the document. It MUST NOT appear in the
```

Description: Requirements with no row in the table: `automation` MUST NOT appear in the frontmatter or the YAML/JSON form (lines 285-287; examples/invalid/short-answer-automation-field.yaml expects schema-error at path `automation`); an implementation SHOULD offer automation as a read-only property (288-289); a question MAY have one `[short-answer/reject]` block (142-143); a `[short-answer]` block MAY have no pattern (221), which has no info row. Other gaps are already reported in F16 and F17.

Impact: The table is not the full list of rules of the type.

Suggested fix: Add an error row (`schema-error`) for `automation`, and either info rows for the MAY permissions or a note that the table lists only diagnostics. Move the read-only property SHOULD out of RFC language or give it a row.

F32: No corpus vectors for feedback selection, `incorrectFeedback`, pre-validation and the `f`/`b` flags

Location: examples/grading/short-answer.yaml:13

```
#   examples: { response: string, score: 0 | 1 | "pending" }[]
```

Description: The grading vectors record only a score. No example exercises: which feedback a response gets (first matching pattern with feedback, accept list for a correct response, short-answer.md:388-397); `incorrectFeedback` for an unmatched or feedback-less rejected response (399-403); pre-validation, including the `preReject` precedence over `preAccept` (113-115); the `f` and `b` flags and an author-written anchor under them (patterns.md:295-302; examples/valid/short-answer/regex-flags-f-b.mdq.md only checks that the document parses); `n` not applying the `letters` table (patterns.md:291-293).

Impact: mdq-py and mdq-js can disagree on these rules with every shared test passing.

Suggested fix: Add a `feedback` key to the grading vectors, a pre-validation vector file, and regex vectors for `f`, `b` and `n`.

F20: The exam `grading` table does not say what it does to short answer

Location: docs/exam.md:335-337

```
Each question type accepts its own set of strategies in its `grading` field
```

Description: Short answer has no `grading` field and its score is binary (short-answer.md:252). The table has no column for it, and the mapping form (exam.md:355-361) has no key for it, but no sentence says that the exam `grading` does not apply, or that a wrong short answer scores 0 and not -1 under `symmetric`, unlike a wrong multiple choice pick.

Impact: A reader can expect `symmetric` to penalize a wrong short answer.

Suggested fix: Add a sentence: types without a `grading` field (short answer, numeric, essay, ordering?) ignore the exam `grading` and score in [0, 1].

F31: Rows of Additional Rules with no text in the spec

Location: docs/question-types/short-answer.md:409, docs/question-types/short-answer.md:420, docs/question-types/short-answer.md:421

```
| incorrectFeedback                    | error    | blank-text             | if defined, must
have at least one visible character                                    |
```

Description: Three rows are not described in the prose of short-answer.md or patterns.md: a pattern list must have at least one pattern (row 409; short-answer.md:86 and patterns.md:154 only say 'an array of patterns'); `incorrectFeedback` must not be blank (row 420; lines 84 and 399-403 do not say it); an implicit anchor written by the author is redundant (row 421; patterns.md:250-256 says anchors are 'supported and optional' and mentions no diagnostic).

Impact: The rules exist only in the table, so authors reading the sections do not learn them.

Suggested fix: Add one sentence for each rule in its section, with the code.

F34: GLOSSARY.md has no entry for pattern, settled, exact or inexact literal, or pre-validation

Location: docs/question-types/short-answer.md:256

```
A response is **settled** when the question decides if it is correct. An
```

Description: short-answer.md and patterns.md define and use these terms in bold or as section titles; GLOSSARY.md has entries for Automation and Pending response but none for the terms that the short-answer type depends on.

Impact: Readers of other types and of the glossary meet `settled` and `pattern` without a definition.

Suggested fix: Add short entries for Pattern, Settled response, Exact literal, Inexact literal and Pre-validation.

Inconsistencies

Severity	Location	Description
MEDIUM	docs/question-types/fill-in.md:210-212, docs/question-types/short-answer.md:152-159	**F28** Fill-in grammar uses an undefined `PATTERN` terminal for what short-answer splits into `exact | inexact`
MEDIUM	docs/question-types/short-answer.md:419	**F30** Row `no-correct-answer` says 'must not' at warning level
LOW	docs/question-types/short-answer.md:301-303, docs/question-types/short-answer.md:379-381	**F33** Two automation examples are not valid documents

F28: Fill-in grammar uses an undefined `PATTERN` terminal for what short-answer splits into `exact | inexact`

Location: docs/question-types/fill-in.md:210-212, docs/question-types/short-answer.md:152-159

```
answer           : PATTERN
```

Description: short-answer.md declares the single-line form as `patterns : content | nl pattern_list` with `content : exact | inexact` and the terminals EXACT_TEXT and PLAIN_TEXT. fill-in.md, which says the same rules apply, declares `patterns : answer | nl pattern_list` with `answer : PATTERN`; PATTERN is defined in neither fill-in.md nor grammar.md. The same construct has two rule names and two definitions.

Impact: Fixes to the short-answer grammar (continuation lines, backtick rules) do not reach fill-in, and a reader cannot compare the two.

Suggested fix: Use the same rules in fill-in.md (`content`, `exact`, `inexact`) or define them once in grammar.md and reference them from both.

F30: Row `no-correct-answer` says 'must not' at warning level

Location: docs/question-types/short-answer.md:419

```
| unmatched                            | warning  | no-correct-answer      | must not be
"incorrect" when the question has no `accept` pattern[^8]              |
```

Description: The rule text uses 'must not' (MUST NOT: error) but the level is warning, and lint-codes.md:35 and examples/valid/short-answer/no-correct-answer.yaml treat the document as valid with a warning. The prose (short-answer.md:266-269, 281) describes `unmatched: incorrect` with no `accept` as a valid automatic question and never says it SHOULD be avoided.

Impact: A reader cannot tell whether the document is invalid.

Suggested fix: Write 'should not' in the row and add a SHOULD NOT sentence in the Grading section.

F33: Two automation examples are not valid documents

Location: docs/question-types/short-answer.md:301-303, docs/question-types/short-answer.md:379-381

Mechanical fix

```
[short-answer]: Brasília
```

Description: The 'Automatic' and 'Manual' examples hold only the tag, with no stem. `mdq validate` rejects both ('parse-error: A required field is missing'); with a stem line they pass. The other four examples of the section are complete questions.

Impact: A reader who copies the example gets a parse error.

Suggested fix: Add the stem `What is the capital of Brazil?` and `Name a biome of central Brazil.`, as in the sibling examples.

Grammar violations

Severity	Location	Description
CRITICAL	docs/question-types/short-answer.md:152-159	**F2** Single-line content has no continuation lines, but the prose and corpus allow them
HIGH	docs/question-types/short-answer.md:150-151	**F1** Short-answer tag rule has no leading ws?, so an indented tag is rejected
HIGH	docs/question-types/short-answer.md:150-152	**F3** Empty tag line followed by a text line: content or epilogue is undecided
MEDIUM	docs/question-types/short-answer.md:153-159	**F4** Grammar has no regex alternative: /-delimited content parses as inexact

Severity	Location	Description
MEDIUM	docs/question-types/short-answer.md:153-159	**F5** Partial backtick content: grammar rejects it as a parse failure, corpus expects schema-error
MEDIUM	docs/question-types/short-answer.md:154-159	**F6** Grammar accepts patterns that become empty after normalization
MEDIUM	docs/question-types/short-answer.md:156	**F7** pattern_item has no grammar, so the pattern line is unconstrained

F2: Single-line content has no continuation lines, but the prose and corpus allow them

Location: docs/question-types/short-answer.md:152-159

```
patterns     : content | nl pattern_list
content      : exact | inexact
exact        : "`" EXACT_TEXT "`" ws?
inexact      : PLAIN_TEXT
```

Description: `content` is one line: EXACT_TEXT and PLAIN_TEXT exclude `\r` and `\n` and nothing follows them. Tested with Lark, all rejected: `[short-answer]: Rio\nNegro`, `[short-answer/reject]: Rio\nAmazonas`, and `` `[short-answer]: math.\nisnan . `` docs/references/patterns.md:223-226 says the content of a single-line block continues on the next lines (Rio + Negro is Rio Negro), short-answer.md:231-233 repeats it, patterns.md:230 says the joining space is significant in an exact literal (so an exact literal may span lines), and the valid example examples/valid/short-answer/continuation-lines.mdq.md uses [short-answer/reject]: Rio followed by Amazonas`.

Impact: A wrapped answer, common when an editor reflows text, is valid by the prose and the corpus but invalid by the grammar. Implementers get two contradictory specs.

Suggested fix: Add continuation lines to `content`, e.g. `inexact : PLAIN_TEXT (nl CONT_LINE)*` with a terminal that excludes blank lines and the starts of interrupting blocks (as LAZY_START does), and let EXACT_TEXT continue across line endings in the same way.

F1: Short-answer tag rule has no leading ws?, so an indented tag is rejected

Location: docs/question-types/short-answer.md:150-151

Mechanical fix

```
accept_block : "[" "short-answer" "/accept"? "]" ws? ":" ws? patterns?
reject_block : "[" "short-answer" "/reject" "]" ws? ":" ws? patterns
```

Description: Neither block rule starts with `ws?`. Tested with Lark: `[short-answer] : Brasília` (three leading spaces) is rejected. The valid corpus example examples/valid/short-answer/inline-id-and-indented-tag.mdq.md uses exactly this tag line and expects `accept: [Brasília]`. CommonMark allows up to three spaces before a paragraph, and docs/question-types/numeric.md:64 starts its tag rule with `ws?`. docs/references/grammar.md:58-61 defines what a leading `ws?` means at block start.

Impact: The grammar contradicts a valid example and the numeric tag rule; an implementation that follows the snippet rejects indented tag lines.

Suggested fix: Prefix both rules with `ws?`, as numeric.md does: `accept_block : ws? "[" "short-answer" ....`

F3: Empty tag line followed by a text line: content or epilogue is undecided

Location: docs/question-types/short-answer.md:150-152

```
accept_block : "[" "short-answer" "/accept"? "]" ws? ":" ws? patterns?
```

Description: For `[short-answer]:\nBrasília` the grammar ends the block after the colon (`patterns?` is empty; `nl` may only precede `pattern_list`), so `Brasília` is not a pattern and the question has no accept list. docs/references/patterns.md:223-225 says 'The lines after the tag line, up to the first blank line, continue the pattern', which reads `Brasília` as the content. In CommonMark the line is a lazy continuation of the same paragraph. For `[short-answer/reject]:\nRio` the grammar rejects the block (no pattern), while the continuation rule gives the pattern `Rio`.

Impact: Two readings give different documents: a manual question with stray text, or an automatic question with accept `Brasília`; for reject, an error or a valid block.

Suggested fix: State whether continuation starts only after a non-empty tag line, and encode it in `patterns` (for example, allow `nl content` or forbid it explicitly with a lint code).

F4: Grammar has no regex alternative: /-delimited content parses as inexact

Location: docs/question-types/short-answer.md:153-159

```
content      : exact | inexact
```

Description: Tested with Lark: `[short-answer]: /[Bb]ras[íi]lia/i` parses as `inexact` (PLAIN_TEXT), and so does the unclosed `[short-answer]: /Bras[íi]lia`. short-answer.md:211-212 says content delimited by `/` is a regex, and the corpus (examples/invalid/short-answer-unclosed-regex-content) expects `invalid-regex` for the unclosed form, which patterns.md:148 requires to be closed. The parse tree labels a regex as an inexact literal and the grammar cannot tell an unclosed regex from a plain literal.

Impact: An implementation that dispatches on the parse tree treats regexes and unclosed regexes as plain literals; the classification lives only in prose.

Suggested fix: Add `regex : "/" REGEX_BODY "/" REGEX_FLAGS? ws?` to `content` and restrict PLAIN_TEXT to not start with `/`, so an unclosed regex fails as the corpus expects.

F5: Partial backtick content: grammar rejects it as a parse failure, corpus expects schema-error

Location: docs/question-types/short-answer.md:153-159

```
exact        : "`" EXACT_TEXT "`" ws?
```

Description: Tested with Lark: `` `[short-answer]: math.isnan (x) and [short-answer]: math.isnan` `` are rejected (PLAIN_TEXT cannot start with a backtick, and exact allows only ws after the closing backtick). A grammar failure reads as parse-error, but examples/invalid/short-answer-partial-backtick.lint.json expects schema-error at path accept/0. The Additional Rules table (short-answer.md:407-422) has no row for this case at all.

Impact: Implementations that follow the grammar report a different code than the corpus; the rule that rejects the input is not stated where the lint codes are listed.

Suggested fix: Add an Additional Rules row for content that opens with a backtick and is not a complete span, with the code the corpus uses, and say in the grammar section that the snippet leaves this case to that rule.

F6: Grammar accepts patterns that become empty after normalization

Location: docs/question-types/short-answer.md:154-159

```
EXACT_TEXT   : /[^`\n\r]+/
PLAIN_TEXT   : /[^` \t\r\n][^\r\n]*/
```

Description: Tested with Lark: `[short-answer]: (exact, one space) and [short-answer]: + U+200B (inexact) are accepted. Exact-literal step 2 (patterns.md:131) trims the space and inexact step 2 (patterns.md:48) removes U+200B, so both patterns compare as the empty string. patterns.md:20 says a pattern string 'cannot be empty' and examples/invalid/short-answer-empty-pattern.yaml rejects ""`, but neither the text nor the grammar says whether 'empty' is checked before or after normalization.

Impact: A pattern that matches only a blank response passes validation, or is rejected, depending on how an implementation reads 'empty'.

Suggested fix: State whether a pattern that is empty after normalization is invalid, and add the code (for example `blank-text`) to the Additional Rules table.

F7: pattern_item has no grammar, so the pattern line is unconstrained

Location: docs/question-types/short-answer.md:156

```
pattern_list : pattern_item+
```

Description: short-answer.md:163-165 sends `pattern_item` to docs/references/patterns.md#pattern-items, which has only prose and no Lark rule. Nothing in the grammar says the pattern line uses `content` (exact | inexact), so it does not reject an empty item (* with only a feedback line, a parse-error in examples/invalid/short-answer-empty-pattern-line) or an item that opens with an unclosed backtick, and it does not model item continuation lines or the observations rule of grammar.md:129-136. The stub had to be invented to test the list form at all.

Impact: The list form, the most used form with feedback, is specified only in prose; the single-line and list forms can drift.

Suggested fix: Define `pattern_item : ws? content (nl CONT_LINE)* nl (blank_line* observations)?` (or equivalent) in patterns.md and reference it from short-answer.md.

Schema mismatches

Severity	Location	Description
MEDIUM	schema/short-answer.yaml:128	**F8** Schema patternString uses \s, which disagrees with PLAIN_TEXT and between Python and JavaScript
MEDIUM	schema/short-answer.yaml:17-20	**F22** Schema says a `[short-answer]` block's content is the first accept entry
MEDIUM	schema/short-answer.yaml:24-28, schema/fill-in.yaml:145-149	**F23** Schema says rejecting never changes a score, but a `reject` match settles a response that would be pending
LOW	schema/short-answer.yaml:123-128, docs/references/patterns.md:20	**F24** Schema rejects a whitespace-only pattern; the spec only forbids an empty one

F8: Schema patternString uses \s, which disagrees with PLAIN_TEXT and between Python and JavaScript

Location: schema/short-answer.yaml:128

```
pattern: "^(?:`[^`\\n]+`[ \\t]*|[^`\\s][\\s\\S]*|\\s+[^`\\s][\\s\\S]*)$"
```

Description: docs/references/grammar.md:11-19 forbids `\s` because Python and JavaScript disagree on it, but the schema pattern that mirrors EXACT_TEXT/PLAIN_TEXT uses it. Tested: U+FEFF + ` x matches in Python re and fails in JavaScript; U+00A0 + x and U+0085 + x fail in both, while PLAIN_TEXT (short-answer.md:159) accepts all three as inexact text, which grammar.md:44-46 confirms for U+00A0. The exact alternative also excludes only \n, so a\rb passes the schema but not EXACT_TEXT ([^\n\r]+). It also rejects spaces before a backtick (" x", a schema-error in mdq-py) while it allows them before / and plain text, and patterns.md:147 strips them for a regex.

Impact: The same content validates in mdq-py and fails in mdq-js, and the schema rejects content the grammar defines as text.

Suggested fix: Replace `\s` with the explicit class `[ \t\r\n]` used by PLAIN_TEXT, exclude `\r` in the exact alternative, and decide if leading spaces before an exact literal are stripped as before a regex.

F22: Schema says a `[short-answer]` block's content is the first `accept` entry

Location: schema/short-answer.yaml:17-20

Mechanical fix

```
any entry. The content of a simple `[short-answer]` block is the
```

Description: The description implies that a single-line `[short-answer]` block adds one entry to a longer `accept` list. The spec says the block and the field are two spellings of the same list (short-answer.md:91-95, 174-176) and that using both is a `conflicting-accept` error (short-answer.md:204-206). It also calls `accept` the canonical form of `[short-answer/accept]`, while the spec's main spelling is `[short-answer]`, with `/accept` as alias (short-answer.md:144).

Impact: A reader of the schema can expect `[short-answer]: X` plus `accept: [Y]` to give `[X, Y]`, which the spec rejects.

Suggested fix: Say: the canonical form of the `[short-answer]` block (alias `[short-answer/accept]`); the single-line form gives a list of one entry.

F23: Schema says rejecting never changes a score, but a `reject` match settles a response that would be pending

Location: schema/short-answer.yaml:24-28, schema/fill-in.yaml:145-149

Mechanical fix

```
the `[short-answer/reject]` block. Rejecting never changes a score
```

Description: Under `unmatched: manual` (explicit, or the default when there is no `accept`), a response that matches a `reject` pattern is incorrect and scores 0, while the same response with no `reject` pattern is pending (short-answer.md:260-264, 374-375; examples/grading/short-answer.yaml 'Reject only': Savana scores 0, Cerrado is pending). The `reject` list also changes the automation from `manual` to `semi-automatic` (short-answer.md:282-283). The fill-in blank schema repeats the sentence.

Impact: Readers of the schema think `reject` is feedback only and miss that it settles responses in semi-automatic questions.

Suggested fix: Say: a response that matches no `accept` pattern and matches a `reject` pattern is incorrect; `reject` decides responses that `unmatched: manual` would leave pending, and attaches feedback.

F24: Schema rejects a whitespace-only pattern; the spec only forbids an empty one

Location: schema/short-answer.yaml:123-128, docs/references/patterns.md:20

```
A pattern string is plain text, never markdown. It cannot be empty.
```

Description: patterns.md:20 says a pattern string cannot be empty. The schema comment says a pattern 'Must carry at least one non-space character' and the regex rejects " ". The spec text never states the non-blank rule, and the Additional Rules table has no row for either.

Impact: The schema enforces a rule that the spec does not declare.

Suggested fix: State in patterns.md that a pattern string must have a character that is not a space or tab, with its code.

Prioritized recommendations

P1: Specify the single-line content, continuation lines and pattern items in the grammar, shared by short answer and fill-in. (Findings: F2, F9, F3, F7, F28)

P1: Pin down regex semantics: subject normalization, Unicode mode, delimiters and the n flag. (Findings: F10, F11, F12, F13)

P2: Specify the pattern-string syntax once and align the schema with the grammar terminals. (Findings: F4, F5, F6, F8, F24)

P2: Complete the Additional Rules table and fix the level wording. (Findings: F16, F17, F29, F30, F31)

P2: Define feedback selection for every pattern list, and add feedback and pre-validation vectors. (Findings: F25, F26, F27, F32)

P2: Define the short-answer response value in responses.md. (Findings: F15)

P3: Decide on exact literals with backticks, the exam grading sentence and the glossary entries. (Findings: F18, F20, F34)

P3: Apply the mechanical fixes: leading ws? in the tag rules, responses.md normalization text, the accept and reject schema descriptions, and the two incomplete automation examples. (Findings: F1, F14, F22, F23, F33)

GitHub issues

Each block below is a complete issue, ready to paste.

````
--- ISSUE 1 ---
# [Spec] Define the single-line content and the pattern item in the grammar

**Labels:** critical, high, medium

## Problem

The Lark snippet allows one line of content, while the prose and the corpus allow continuation
lines; the prose says every line up to a blank line continues the pattern, while mdq-py ends it
 at any line that interrupts a CommonMark paragraph; an empty tag followed by a text line has
two readings; pattern_item has no grammar; fill-in uses a different, undefined PATTERN
terminal. Recommended: define `content` with continuation lines that stop at the same lines
LAZY_START excludes, define `pattern_item` in patterns.md, and reference both from short-
answer.md and fill-in.md. Decide what a `>`/`!` line after single-line content is (recommended:
 parse-error, since single-line content has no feedback).

## Evidence

- `docs/question-types/short-answer.md:152-159`: Single-line content has no continuation lines,
 but the prose and corpus allow them

  ```lark
  patterns     : content | nl pattern_list
  content      : exact | inexact
  exact        : "`" EXACT_TEXT "`" ws?
  inexact      : PLAIN_TEXT
  ```
- `docs/references/patterns.md:223-226, docs/question-types/short-answer.md:182-183`: Single-
line content: which lines continue the pattern is unclear

  ```
  The content of a single-line block continues in the same way. The lines after
  the tag line, up to the first blank line, continue the pattern:
  ```
- `docs/question-types/short-answer.md:150-152`: Empty tag line followed by a text line:
content or epilogue is undecided

  ```lark
  accept_block : "[" "short-answer" "/accept"? "]" ws? ":" ws? patterns?
  ```
- `docs/question-types/short-answer.md:156`: pattern_item has no grammar, so the pattern line
is unconstrained

  ```lark
  pattern_list : pattern_item+
  ```
- `docs/question-types/fill-in.md:210-212, docs/question-types/short-answer.md:152-159`: Fill-
in grammar uses an undefined `PATTERN` terminal for what short-answer splits into `exact |
inexact`

  ```
  answer           : PATTERN
  ```


## Impact

- **Single-line content has no continuation lines, but the prose and corpus allow them:** A
wrapped answer, common when an editor reflows text, is valid by the prose and the corpus but
invalid by the grammar. Implementers get two contradictory specs.
- **Single-line content: which lines continue the pattern is unclear:** Two implementations can
 produce different answer keys from the same document. An author who writes feedback under a
single-line answer gets it in the epilogue, shown to every student, with no diagnostic.
- **Empty tag line followed by a text line: content or epilogue is undecided:** Two readings
give different documents: a manual question with stray text, or an automatic question with
accept `Brasília`; for reject, an error or a valid block.
- **pattern_item has no grammar, so the pattern line is unconstrained:** The list form, the
most used form with feedback, is specified only in prose; the single-line and list forms can
drift.
- **Fill-in grammar uses an undefined `PATTERN` terminal for what short-answer splits into
`exact | inexact`:** Fixes to the short-answer grammar (continuation lines, backtick rules) do
````

```
not reach fill-in, and a reader cannot compare the two.

## Suggested fix

- **Single-line content has no continuation lines, but the prose and corpus allow them:** Add
continuation lines to `content`, e.g. `inexact : PLAIN_TEXT (nl CONT_LINE)*` with a terminal
that excludes blank lines and the starts of interrupting blocks (as LAZY_START does), and let
EXACT_TEXT continue across line endings in the same way.
- **Single-line content: which lines continue the pattern is unclear:** State which lines end
the single-line content: the same lines that LAZY_START excludes (grammar.md, Feedback and
comment lines). Decide what a `>` or `!` line right after single-line content is: an error, a
warning, or feedback for that pattern.
- **Empty tag line followed by a text line: content or epilogue is undecided:** State whether
continuation starts only after a non-empty tag line, and encode it in `patterns` (for example,
allow `nl content` or forbid it explicitly with a lint code).
- **pattern_item has no grammar, so the pattern line is unconstrained:** Define `pattern_item :
 ws? content (nl CONT_LINE)* nl (blank_line* observations)?` (or equivalent) in patterns.md and
 reference it from short-answer.md.
- **Fill-in grammar uses an undefined `PATTERN` terminal for what short-answer splits into
`exact | inexact`:** Use the same rules in fill-in.md (`content`, `exact`, `inexact`) or define
 them once in grammar.md and reference them from both.

## Acceptance criteria

- [ ] The Lark snippets accept examples/valid/short-answer/continuation-lines.mdq.md
- [ ] patterns.md has a Lark rule for pattern_item used by short-answer.md and fill-in.md
- [ ] The spec states the result of `[short-answer]: X` followed by a `>` line and by an empty-
tag-then-text input, with a corpus example for each
- [ ] scripts/spec_audit.py passes

--- END ISSUE 1 ---
```

````
--- ISSUE 2 ---
# [Spec] Specify the pattern-string syntax once: regex, exact, blank

**Labels:** medium, low

## Problem

The grammar has no regex alternative, a partial backtick span fails the grammar but the corpus
expects schema-error, patterns empty after normalization are accepted, and the schema regex
uses `\s` and differs from EXACT_TEXT and PLAIN_TEXT. Recommended: add a `regex` alternative to
 `content`, state that a pattern must have a visible character after normalization (code
`blank-text` or `schema-error`), and rewrite the schema pattern with explicit classes that
match the terminals.

## Evidence

- `docs/question-types/short-answer.md:153-159`: Grammar has no regex alternative:
`/`-delimited content parses as inexact

  ```lark
  content      : exact | inexact
  ```
- `docs/question-types/short-answer.md:153-159`: Partial backtick content: grammar rejects it
as a parse failure, corpus expects schema-error

  ```lark
  exact        : "`" EXACT_TEXT "`" ws?
  ```
- `docs/question-types/short-answer.md:154-159`: Grammar accepts patterns that become empty
after normalization

  ```lark
  EXACT_TEXT   : /[^`\n\r]+/
  PLAIN_TEXT   : /[^` \t\r\n][^\r\n]*/
  ```
- `schema/short-answer.yaml:128`: Schema patternString uses \s, which disagrees with PLAIN_TEXT
 and between Python and JavaScript

  ```yaml
  pattern: "^(?:`[^`\\n]+`[ \\t]*|[^`\\s][\\s\\S]*|\\s+[^`\\s][\\s\\S]*)$"
  ```
- `schema/short-answer.yaml:123-128, docs/references/patterns.md:20`: Schema rejects a
whitespace-only pattern; the spec only forbids an empty one

  ```
  A pattern string is plain text, never markdown. It cannot be empty.
  ```

## Impact

- **Grammar has no regex alternative: `/`-delimited content parses as inexact:** An
implementation that dispatches on the parse tree treats regexes and unclosed regexes as plain
literals; the classification lives only in prose.
- **Partial backtick content: grammar rejects it as a parse failure, corpus expects schema-
error:** Implementations that follow the grammar report a different code than the corpus; the
rule that rejects the input is not stated where the lint codes are listed.
- **Grammar accepts patterns that become empty after normalization:** A pattern that matches
only a blank response passes validation, or is rejected, depending on how an implementation
reads 'empty'.
- **Schema patternString uses \s, which disagrees with PLAIN_TEXT and between Python and
JavaScript:** The same content validates in mdq-py and fails in mdq-js, and the schema rejects
content the grammar defines as text.
- **Schema rejects a whitespace-only pattern; the spec only forbids an empty one:** The schema
enforces a rule that the spec does not declare.

## Suggested fix

- **Grammar has no regex alternative: `/`-delimited content parses as inexact:** Add `regex :
"/" REGEX_BODY "/" REGEX_FLAGS? ws?` to `content` and restrict PLAIN_TEXT to not start with
`/`, so an unclosed regex fails as the corpus expects.
````

```
- **Partial backtick content: grammar rejects it as a parse failure, corpus expects schema-
error:** Add an Additional Rules row for content that opens with a backtick and is not a
complete span, with the code the corpus uses, and say in the grammar section that the snippet
leaves this case to that rule.
- **Grammar accepts patterns that become empty after normalization:** State whether a pattern
that is empty after normalization is invalid, and add the code (for example `blank-text`) to
the Additional Rules table.
- **Schema patternString uses \s, which disagrees with PLAIN_TEXT and between Python and
JavaScript:** Replace `\s` with the explicit class `[ \t\r\n]` used by PLAIN_TEXT, exclude `\r`
 in the exact alternative, and decide if leading spaces before an exact literal are stripped as
 before a regex.
- **Schema rejects a whitespace-only pattern; the spec only forbids an empty one:** State in
patterns.md that a pattern string must have a character that is not a space or tab, with its
code.

## Acceptance criteria

- [ ] `content` has exact, regex and inexact alternatives
- [ ] The schema patternString regex uses only explicit classes and agrees with EXACT_TEXT and
PLAIN_TEXT
- [ ] A pattern of only U+200B or a blank exact literal has a documented code and a corpus
example

--- END ISSUE 2 ---
```

````
--- ISSUE 3 ---
# [Spec] Pin down regex semantics: subject, Unicode mode, delimiters and the n flag

**Labels:** high, medium

## Problem

The spec does not say how the response is prepared before a regex match (NFC? trim?), which
Unicode mode a regex runs in (`.`, `i`, `\u{...}`), how `\d`/`\w`/`\s` behave in Python, which
`/` closes the regex, whether spaces after flags are allowed, or how `n` treats escapes in the
regex source. Recommended: NFC plus trim of UNICODE_SPACE on the subject, code-point mode with
simple case folding, ASCII shorthand classes, the last `/` closes, trailing spaces and tabs
stripped, and `n` strips only the subject with a lint for an `Mn` code point left in the regex.

## Evidence

- `docs/references/patterns.md:145-150`: The response is not normalized or trimmed before a
regex match

  ```
  A pattern is a regex if it starts with a `/` after stripping spaces and tabs.
  ```
- `docs/references/patterns.md:260-267, docs/references/patterns.md:304-305`: Regex Unicode
mode, case folding and shorthand classes are undefined

  ```
  * Unicode: Unicode characters are allowed in the regex. Unicode escape sequences
  ```
- `docs/references/patterns.md:147-150, docs/references/patterns.md:31-32`: Regex delimiters:
an unescaped `/` in the body and spaces after the flags

  ```
  A regex must be closed by a `/` as well, and may include optional flags after
  ```
- `docs/references/patterns.md:289-294`: The `n` flag strips diacritics from the regex source
with no rule for escapes

  ```
  * `n`: Strip diacritics from both the pattern and the response before
  ```

## Impact

- **The response is not normalized or trimmed before a regex match:** A student who types a
trailing space or newline, or whose input method produces NFD, fails a regex answer that a
literal would accept. Implementations can disagree.
- **Regex Unicode mode, case folding and shorthand classes are undefined:** The same regex
accepts different responses in mdq-py and mdq-js.
- **Regex delimiters: an unescaped `/` in the body and spaces after the flags:** Authors cannot
 predict whether `/` needs escaping; implementations that follow JavaScript literal syntax
reject documents mdq-py accepts.
- **The `n` flag strips diacritics from the regex source with no rule for escapes:** A regex
with `n` can match nothing, with no diagnostic, and implementations can differ.

## Suggested fix

- **The response is not normalized or trimmed before a regex match:** Define the regex subject,
 for example: normalize the response to NFC and remove UNICODE_SPACE from both ends, as for
exact literals. Add grading vectors for it.
- **Regex Unicode mode, case folding and shorthand classes are undefined:** Pick one mode
(recommended: code points, as JavaScript `u`/`v` mode and Python `re` do) and state the
semantics of `.`, `i` (simple case folding?) and the shorthand classes (ASCII only, as
JavaScript). Add vectors to examples/ for each.
- **Regex delimiters: an unescaped `/` in the body and spaces after the flags:** State that the
 last `/` closes the regex and that an unescaped `/` in the body is allowed (or forbidden,
`invalid-regex`). State that spaces and tabs after the flags are stripped.
- **The `n` flag strips diacritics from the regex source with no rule for escapes:** Define `n`
 on the subject only (strip the response) and require the regex to be written without
diacritics, or define how escapes are treated. A lint for a regex under `n` that still holds an
 `Mn` code point would catch the mistake.
````

```
## Acceptance criteria

- [ ] patterns.md lists the steps applied to the response before a regex match
- [ ] patterns.md states the Unicode mode, the case folding of `i` and the shorthand classes
- [ ] examples/ has regex grading vectors for trimming, NFD input, astral `.`, `\d` on non-
ASCII digits and `n` with an escape

--- END ISSUE 3 ---
```

````
--- ISSUE 4 ---
# [Spec] Complete the Additional Rules table of short answer

**Labels:** medium, low

## Problem

The required `[short-answer]` tag, blank pattern strings, partial backtick spans, empty pattern
 lines, a `[short-answer/preAccept]` block and the `automation` key have no row; `no-correct-
answer` says 'must not' at warning level; three rows have no text. The required tag also blocks
 a frontmatter-only accept list. Recommended: keep the tag required but state that an empty tag
 with an `accept` field is valid, add one row per diagnostic with the code the corpus uses, and
 fix the level wording.

## Evidence

- `docs/question-types/short-answer.md:140-145, docs/question-types/short-answer.md:204-206`:
The required `[short-answer]` tag has no rule row and blocks the frontmatter-only form

  ```
  incorrect responses. A question MUST have a `[short-answer]` block and MAY
  ```
- `docs/question-types/short-answer.md:407-422`: Additional Rules misses the pattern-string and
 preAccept-block rules

  ```
  | accept, reject, preAccept, preReject | error    | schema-error           | a pattern list,
when present, has at least one pattern (schema)                     |
  ```
- `docs/question-types/short-answer.md:285-289, docs/question-types/short-answer.md:142-143,
docs/question-types/short-answer.md:221`: Additional Rules has no row for the `automation` key,
 the MAY permissions and the read-only property SHOULD

  ```
  `automation` is not a field of the document. It MUST NOT appear in the
  ```
- `docs/question-types/short-answer.md:419`: Row `no-correct-answer` says 'must not' at warning
 level

  ```
  | unmatched                            | warning  | no-correct-answer      | must not be
"incorrect" when the question has no `accept` pattern[^8]               |
  ```
- `docs/question-types/short-answer.md:409, docs/question-types/short-answer.md:420,
docs/question-types/short-answer.md:421`: Rows of Additional Rules with no text in the spec

  ```
  | incorrectFeedback                    | error    | blank-text             | if defined, must
 have at least one visible character                                    |
  ```

## Impact

- **The required `[short-answer]` tag has no rule row and blocks the frontmatter-only form:**
Authors who use the frontmatter form get an unexplained parse error, and the empty tag that
fixes it looks like a 'no pattern' question.
- **Additional Rules misses the pattern-string and preAccept-block rules:** The table is not
the complete list of diagnostics; an implementer reading it misses four errors.
- **Additional Rules has no row for the `automation` key, the MAY permissions and the read-only
 property SHOULD:** The table is not the full list of rules of the type.
- **Row `no-correct-answer` says 'must not' at warning level:** A reader cannot tell whether
the document is invalid.
- **Rows of Additional Rules with no text in the spec:** The rules exist only in the table, so
authors reading the sections do not learn them.

## Suggested fix

- **The required `[short-answer]` tag has no rule row and blocks the frontmatter-only form:**
Either drop the requirement when `type` or `accept`/`reject` is in the frontmatter, or keep it
and add a row (`parse-error`) plus a sentence that an empty tag with an `accept` field is
````

```
valid.
- **Additional Rules misses the pattern-string and preAccept-block rules:** Add one row per
rule, with its code. Decide if the SHOULD at patterns.md:231 gets a warning code or becomes
plain advice.
- **Additional Rules has no row for the `automation` key, the MAY permissions and the read-only
 property SHOULD:** Add an error row (`schema-error`) for `automation`, and either info rows
for the MAY permissions or a note that the table lists only diagnostics. Move the read-only
property SHOULD out of RFC language or give it a row.
- **Row `no-correct-answer` says 'must not' at warning level:** Write 'should not' in the row
and add a SHOULD NOT sentence in the Grading section.
- **Rows of Additional Rules with no text in the spec:** Add one sentence for each rule in its
section, with the code.

## Acceptance criteria

- [ ] Every diagnostic in examples/invalid/short-answer-* has a row
- [ ] Every row has a sentence in the prose
- [ ] Each row uses must/should matching its level

--- END ISSUE 4 ---
```

````
--- ISSUE 5 ---
# [Spec] Define feedback selection for every pattern list and test it

**Labels:** medium

## Problem

patterns.md and the schema say feedback comes from the pattern that 'decided' the response,
while short-answer.md uses the first matching pattern with feedback; preAccept/preReject
feedback has no rule; fill-in blanks have no incorrectFeedback and the spec does not say so; no
 corpus vector checks feedback, pre-validation or the f/b/n flags. Recommended: keep the short-
answer.md rule and reference it, add a pre-validation feedback rule, state that blanks have no
incorrectFeedback, and add a `feedback` key to the grading vectors.

## Evidence

- `docs/references/patterns.md:166-167, schema/short-answer.yaml:108, docs/question-
types/short-answer.md:388-392`: Pattern feedback is said to come from the pattern that
'decided' the response, but the rule picks the first matching pattern that has feedback

  ```
  Only `pattern` is required. `feedback` is shown to the student whose response
  ```
- `docs/question-types/short-answer.md:104-115, docs/question-types/fill-in.md:57-60`: Feedback
 of `preAccept` and `preReject` patterns has no rule

  ```
  systems should warn students about the invalidity of their answer before
  ```
- `docs/question-types/fill-in.md:202-204, docs/question-types/fill-in.md:280-286, schema/fill-
in.yaml:120-128`: Short answer blanks of fill-in have no `incorrectFeedback`, and the spec does
 not say so

  ```
  A short answer blank takes the same answer machinery as a standalone
  ```
- `examples/grading/short-answer.yaml:13`: No corpus vectors for feedback selection,
`incorrectFeedback`, pre-validation and the `f`/`b` flags

  ```
  #   examples: { response: string, score: 0 | 1 | "pending" }[]
  ```

## Impact

- **Pattern feedback is said to come from the pattern that 'decided' the response, but the rule
 picks the first matching pattern that has feedback:** Implementations can show or omit
feedback for the same response.
- **Feedback of `preAccept` and `preReject` patterns has no rule:** Authors write pre-
validation feedback that implementations may never show, or show differently.
- **Short answer blanks of fill-in have no `incorrectFeedback`, and the spec does not say so:**
 A reader cannot tell whether an unmatched blank response can carry feedback; the 'same
machinery' sentence suggests it can.
- **No corpus vectors for feedback selection, `incorrectFeedback`, pre-validation and the
`f`/`b` flags:** mdq-py and mdq-js can disagree on these rules with every shared test passing.

## Suggested fix

- **Pattern feedback is said to come from the pattern that 'decided' the response, but the rule
 picks the first matching pattern that has feedback:** In patterns.md and the schema, refer to
the selection rule of short-answer.md ('Feedback') instead of 'the pattern that decided'.
- **Feedback of `preAccept` and `preReject` patterns has no rule:** Add a rule: when a
submission is invalid, show the feedback of the first matching `preReject` pattern that has
one; state what is shown for a `preAccept` failure (no pattern matched, so no pattern
feedback). Write the warning requirement with SHOULD.
- **Short answer blanks of fill-in have no `incorrectFeedback`, and the spec does not say so:**
 State that blanks have no `incorrectFeedback`, or add the field (per blank or per question).
- **No corpus vectors for feedback selection, `incorrectFeedback`, pre-validation and the
`f`/`b` flags:** Add a `feedback` key to the grading vectors, a pre-validation vector file, and
 regex vectors for `f`, `b` and `n`.
````

````
## Acceptance criteria

- [ ] patterns.md and the schema reference the selection rule of short-answer.md
- [ ] The spec states which feedback an invalid submission shows
- [ ] examples/grading/short-answer.yaml checks feedback; a pre-validation vector file exists

--- END ISSUE 5 ---


--- ISSUE 6 ---
# [Spec] Define the short-answer response value

**Labels:** medium

## Problem

responses.md does not say that a non-string response is malformed, how an empty or whitespace-
only string is graded, or whether line breaks are allowed. Recommended: a string is required
(anything else is a ResponseError), an empty string is graded like any other response, and line
 breaks are allowed.

## Evidence

- `docs/responses.md:148-153`: Short-answer response: type, empty string and line breaks are
not defined

  ```
  The text the student wrote, before normalization.
  ```

## Impact

A YAML response `1960` can be coerced to text in one implementation and refused in another. An
empty submission scores 0 under `unmatched: incorrect` but its status is not specified.

## Suggested fix

Add: the response is a string; any other value is a malformed response (`ResponseError`). State
 that an empty or whitespace-only string is graded like any other response (or is a skip). Add
vectors.

## Acceptance criteria

- [ ] responses.md states the three rules
- [ ] examples/grading/short-answer.yaml has vectors for an empty string under unmatched:
incorrect

--- END ISSUE 6 ---
````

````
--- ISSUE 7 ---
# [Spec] Allow an exact literal that contains a backtick

**Labels:** medium

## Problem

EXACT_TEXT excludes the backtick, so answers such as `` `date` `` cannot be matched exactly.
Options: (a) accept a CommonMark code span of N backticks (recommended, matches CommonMark);
(b) document the limitation and point to a regex.

## Evidence

- `docs/question-types/short-answer.md:158`: An exact literal cannot contain a backtick

  ```
  EXACT_TEXT   : /[^`\n\r]+/
  ```

## Impact

Questions about shells, Markdown or SQL identifiers cannot use exact matching; the error
message does not help.

## Suggested fix

Accept a CommonMark code span of N backticks, with the same one-space stripping, or document
the limitation and suggest a regex.

## Acceptance criteria

- [ ] The grammar and the schema accept the chosen form, or the limitation is documented
- [ ] A corpus example covers it

--- END ISSUE 7 ---


--- ISSUE 8 ---
# [Spec] Say how the exam grading applies to short answer

**Labels:** low

## Problem

The exam grading table has no column for types without a `grading` field. Recommended: one
sentence that short answer, numeric, essay and ordering ignore the exam `grading` and score in
[0, 1].

## Evidence

- `docs/exam.md:335-337`: The exam `grading` table does not say what it does to short answer

  ```
  Each question type accepts its own set of strategies in its `grading` field
  ```

## Impact

A reader can expect `symmetric` to penalize a wrong short answer.

## Suggested fix

Add a sentence: types without a `grading` field (short answer, numeric, essay, ordering?)
ignore the exam `grading` and score in [0, 1].

## Acceptance criteria

- [ ] exam.md states it

--- END ISSUE 8 ---
````

````
--- ISSUE 9 ---
# [Spec] Add short-answer terms to the glossary

**Labels:** low

## Problem

Pattern, settled response, exact literal, inexact literal and pre-validation are not in
GLOSSARY.md.

## Evidence

- `docs/question-types/short-answer.md:256`: GLOSSARY.md has no entry for pattern, settled,
exact or inexact literal, or pre-validation

  ```
  A response is **settled** when the question decides if it is correct. An
  ```

## Impact

Readers of other types and of the glossary meet `settled` and `pattern` without a definition.

## Suggested fix

Add short entries for Pattern, Settled response, Exact literal, Inexact literal and Pre-
validation.

## Acceptance criteria

- [ ] GLOSSARY.md has the five entries

--- END ISSUE 9 ---
````

````
--- ISSUE 10 ---
# Mechanical fixes

**Labels:** mechanical, high, medium, low

## Problem

Each fix below follows from its finding and needs no design decision, so they can be applied
together in one pass and reviewed by reading the diff. Severity still applies: a mechanical fix
 can matter a lot.

### F1 (high): Short-answer tag rule has no leading ws?, so an indented tag is rejected

`docs/question-types/short-answer.md:150-151`

```lark
accept_block : "[" "short-answer" "/accept"? "]" ws? ":" ws? patterns?
reject_block : "[" "short-answer" "/reject" "]" ws? ":" ws? patterns
```

**Suggested fix:** Prefix both rules with `ws?`, as numeric.md does: `accept_block : ws? "["
"short-answer" ...`.

### F14 (medium): responses.md says every short answer is case folded before matching

`docs/responses.md:156-158`

```
Short answers are normalized before matching -- case folded, Unicode
```

**Suggested fix:** Say that each pattern normalizes the response in its own way, with a link to
 patterns.md.

### F22 (medium): Schema says a `[short-answer]` block's content is the first `accept` entry

`schema/short-answer.yaml:17-20`

```
any entry. The content of a simple `[short-answer]` block is the
```

**Suggested fix:** Say: the canonical form of the `[short-answer]` block (alias `[short-
answer/accept]`); the single-line form gives a list of one entry.

### F23 (medium): Schema says rejecting never changes a score, but a `reject` match settles a
response that would be pending

`schema/short-answer.yaml:24-28, schema/fill-in.yaml:145-149`

```
the `[short-answer/reject]` block. Rejecting never changes a score
```

**Suggested fix:** Say: a response that matches no `accept` pattern and matches a `reject`
pattern is incorrect; `reject` decides responses that `unmatched: manual` would leave pending,
and attaches feedback.

### F33 (low): Two automation examples are not valid documents

`docs/question-types/short-answer.md:301-303, docs/question-types/short-answer.md:379-381`

```
[short-answer]: Brasília
```

**Suggested fix:** Add the stem `What is the capital of Brazil?` and `Name a biome of central
Brazil.`, as in the sibling examples.

## Acceptance criteria
````

```
- [ ] F1: Short-answer tag rule has no leading ws?, so an indented tag is rejected
- [ ] F14: responses.md says every short answer is case folded before matching
- [ ] F22: Schema says a `[short-answer]` block's content is the first `accept` entry
- [ ] F23: Schema says rejecting never changes a score, but a `reject` match settles a response
 that would be pending
- [ ] F33: Two automation examples are not valid documents

--- END ISSUE 10 ---
```